

Linked List Data Structures

What is linked list ?

- A node-based database where each node contain a value and a reference to the next or previous node
- Unlike [array](#), linked lists allow dynamic insertion and deletion with constant time complexity, and do not require shifting indices when modifying the list.

Type of Linked list ?

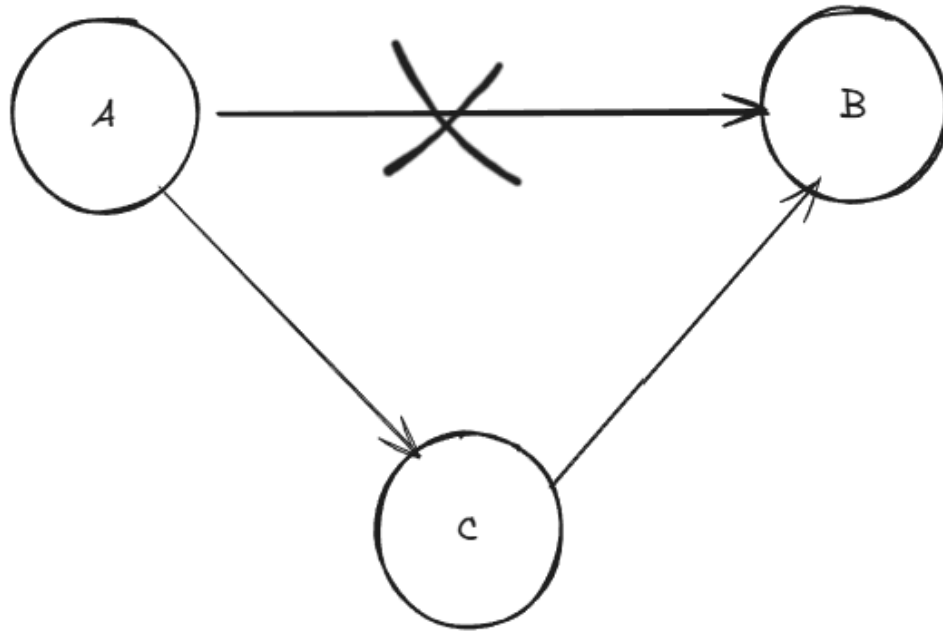
- Singly linked list: only go in one direction, cannot go back if lose reference to a previous node
- Doubly linked list: allow bidirectional traversal, can move forward and backward

Operation of Linked list ?

- Insertion:
 - Traverse to specific node: maybe $O(n)$ but $O(1)$ when head or tail

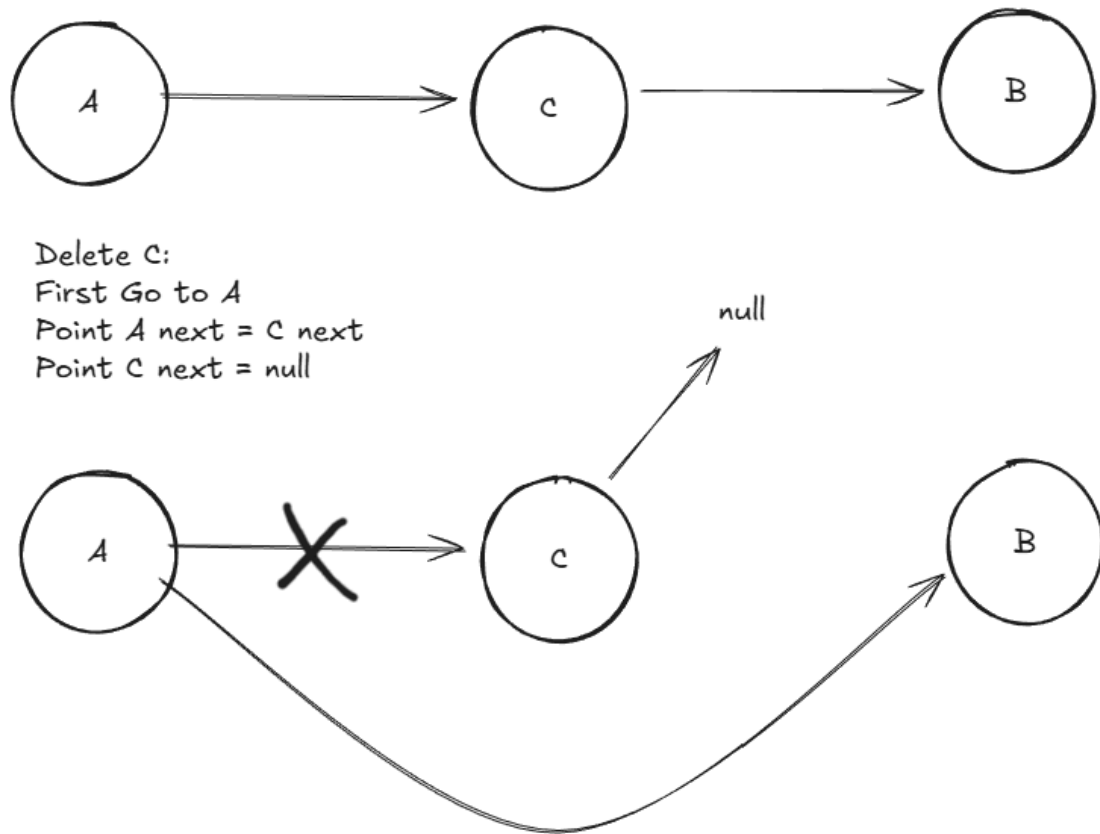
- Modify reference (example in singly linked list), which cost $O(1)$

Insert C after A and before B:
First Go to A
Point C next = A next
Point A next = C



- Deletion:
 - Traverse to specific node: maybe $O(n)$ but $O(1)$ when head or tail

- Modify reference (example in singly linked list), which cost $O(1)$



- Always remember to check if the reference exists

Memory

- Linked lists use heap-allocated objects, which means nodes are stored in memory locations that are typically more expensive than stack memory. Each node is a separate object containing a value and references to other nodes.
- By not contiguous in memory, traversal in Linked List can be costly and they lack the memory optimization benefits of contiguous data structures